Validation of 33 Empty Tables

You just ran an explicit check on the live database and found these 33 tables completely empty:

```
candidates, chat_history, clients, employee_performance_snapshots,
employee_skills, employee_stage_history, extension_requests, feedback, interviews,
invoice_emails, invoice_items, invoice_templates, invoices, job_descriptions,
jobs, leave_audit_logs, offers, organization_memberships, project_time_logs,
project_work_summary, task_evidence, task_feedback, task_reviews,
task_stage_history, task_stages, task_submission_skills, task_submissions,
team_financials, teams, time_logs, timesheets, v_resource_gaps,
view_assignable_employees
```

Here is my validation and recommendation on **why they are empty** and **if it is safe to drop them.**

🟢 Category 1: Entirely Unused Modules (Safe to Delete)

These tables belong to isolated business features (Hiring/ATS and B2B Invoicing) that your app clearly isn't using yet, since zero records have ever been populated.

If you aren't building an ATS or Billing tool right now, DROP these:

- `candidates`, `interviews`, `job_descriptions`, `jobs`, `offers` (Hiring Module)
- `clients`, `invoice_emails`, `invoice_items`, `invoice_templates`, `invoices` (Billing Module)
- `team_financials` (Financial tracking sub-module)

🟢 Category 2: Redundant Logic / Too Granular (Safe to Consolidate or Delete)

Many of these are overly complex tracking layers that were probably built out during early schema design but never actually used in production logic.

- `chat_history`: (If `messages` handles your chat, this is a redundant backup log table. Drop it).
- `leave_audit_logs`: (Just tracking if a leave was 'approved/rejected'. You don't need a dedicated audit log table for this unless for enterprise compliance. Drop it).
- `organization_memberships`: (You likely query `profiles.org_id` instead to map users. Drop it).
- `employee_stage_history` & `task_stage_history`: (State changes. Likely unnecessary bloat. Drop them).
- `extension_requests`: (Probably just an untracked edge case feature. Safe to drop).
- `feedback`, `task_feedback`, `task_reviews`: (If you don't have a formal UI for managers writing 360-reviews on tasks, these are unused. Drop them).
- `employee_skills` & `task_submission_skills`: (Likely bypassed entirely in favor of JSONB tracking on the profiles themselves. Drop them).
- `time_logs`, `timesheets`, `project_time_logs`: (If no one is actively logging explicit hourly punches on projects, this entire Time Tracking module is dead. Keep `attendance` instead and drop these 3).

🟤 Category 3: IMPORTANT WARNINGS (Do NOT drop these yet!)

Wait—these tables are critical to standard workflows. The fact that they are empty means either **a feature in your app isn't working**, or **you just haven't tested it yet.**

Do not delete these until you know why they are empty:

1. **`teams`**: If this is empty, how are you organizing departments? Make sure your app doesn't require a `team_id` constraint.
2. **`task_submissions`**: This is a HUGE warning sign. If `tasks` are being completed, but `task_submissions` is empty, your task-completion workflow/triggers are broken. (The app logic expects points to be calculated on `task_submissions`).
3. **`task_stages`**: If task workflows depend on these stages existing to change state, keep it (or initialize default rows).
4. **`task_evidence`**: If users are supposed to upload screenshots when completing a task, where are they going if this is empty?

Views:

- `project_work_summary`, `v_resource_gaps`, `view_assignable_employees`: These are virtual. They are empty only because the underlying data they look up is empty. Keep them; Views don't take up storage space anyway.

Final Validation Summary

Out of those 33 empty tables:

- **You can immediately and safely delete 24 of them** (Hiring arrays, Invoicing arrays, and redundant Audit logs).
- **You must investigate the 4 core workflow tables** (`task_submissions`, `teams`, `task_stages`, `task_evidence`) before dropping them, because their emptiness likely indicates a bug in your app logic rather than redundancy.